Middle East Technical University Northern Cyprus Campus
CNG445 Software Development with Scripting Languages
Assignment 1

This assignment aims to help you practice Python basics, including objects, primitive types, variables, operators, decision making, loops, sequences, dictionaries, functions, modules, regular expressions, exception handling, command line arguments, and file processing. Your main task in this assignment is to develop a system for analysing the data recorded by a dealership company that sells used cars.

Overview

Most companies try to record any data about their products as possible. By analyzing collected data, they can make better-guided decisions in the future. Suppose you are employed at a company that has many car dealership stores which sell used cars. You are tasked with designing a system that displays some statistics about the dealership stores.

The application you will develop will process two **txt** data files. You will be provided with two input files:

1. **cars.txt** that contains information regarding the cars.
2. **dealerships.txt** that contains information regarding the dealerships.

Sample data for the **cars.txt** file will look like:

ID;brand;properties;MaintenanceNeeds;Revenue;Availability;Days;dealershipID

- **ID**: a 4 digit number that is unique to each car.
- **Brand**: a string of characters can be of any length.
- **Properties**: a string of characters that are separated with the '-' symbol for example; LowBudget-LowKilometer-ParkingCamera-Sensors. There can be one or many properties for a given car.
- **MaintenanceNeeds**: a string of characters of any length for a part of the car that may need maintenance followed by colons ':' and a string of characters for the severity level separated from each other by commas. The following is the format MaintenanceNeed1:SeverityLevelForNeed,MaintenanceNeed2:SeverityLevelForNeed,MaintenanceNeed3:SeverityLevelForNeed,...
- **Revenue**: a number with any number of digits.
- **Availability**: a string of characters that can be sold, rented or available.
- **Days**: a number with any number of digits. This represents a number of rental days if a car is rented.
- **dealershipID**: 3 digit number representing which dealership it belongs to.

Sample data for the **dealerships.txt** file will look like:

ID;Region;ManagerCode

- **ID**: a 3 digit number that is unique to each dealership.
- **Region**: region should be made up of two characters.
- **ManagerCode**: code that refers to the manager responsible for the dealership

You must use Regular Expressions to parse the files, extract the data, and filter out any lines which do not fit the formats given above. After extracting the data you must convert your data to a dataframe to use the pandas library (https://pandas.pydata.org/). You must do the following operations for the cars data: Cars that have the 'LowBudget' property with a higher revenue than 11000 should have their revenue replaced with the average of car revenue which are Low Budget and have their revenue less than 11000.

An example for how to use pandas library is provided below, please keep in mind that you will need to install the pandas library first:

```
import pandas as pd

# assume that we have data as a list of dictionaries such as this one
# as you can see in this example, dictionary keys act as the column names
for the data which is what allows the dataframe conversion
# Please note that this is just an example, and how you choose to represent
your data for your example can look different
# You can look into the pandas documentation or other sources to see
different ways to convert data into a dataframe

data = [
    {'Name': 'Alice', 'Department': 'HR', 'Salary': 50000},
    {'Name': 'Bob', 'Department': 'IT', 'Salary': 55000},
    {'Name': 'Charlie', 'Department': 'IT', 'Salary': 75000},
    {'Name': 'David', 'Department': 'Sales', 'Salary': 60000},
    {'Name': 'Eve', 'Department': 'IT', 'Salary': 52000},
]

# it can be converted into a dataframe like this
df = pd.DataFrame(data)

# check the dataframe by printing to the console
print(df)

# and you can check a single column like following:
print(df['Name'])

# add condition to only select the rows that meet the condition
print(df[df['Salary'] > 60000]['Name'])

# you can easily take the average of a whole column like this
target_average = df[df['Department'] == 'IT']['Salary'].mean()
print(f"Average IT Salary: {target_average}\n")

# loc function can be used with a condition to update only the values that
meet the condition
condition = (df['Department'] == 'IT') & (df['Salary'] < 60000)
df.loc[condition, 'Salary'] = int(target_average)

print("Updated DataFrame:\n", df)
```

When using the data to create your objects you can change the dataframe to a dictionary using the function 'to_dict'.

Implementation Requirements

This application should receive the data file names as a command-line argument and then create the appropriate object structures. Please note that the names and content of the data files can be changed but their formats will be the same. You need to name your main

program as dealershipanalyser.py. The first argument should be the cars file, and the second argument is the dealerships file. The sample command-line execution of the program should be as follows.:

```
python dealershipanalyser.py cars.txt dealership.txt
```

The required object class structure is presented in Figure 1.

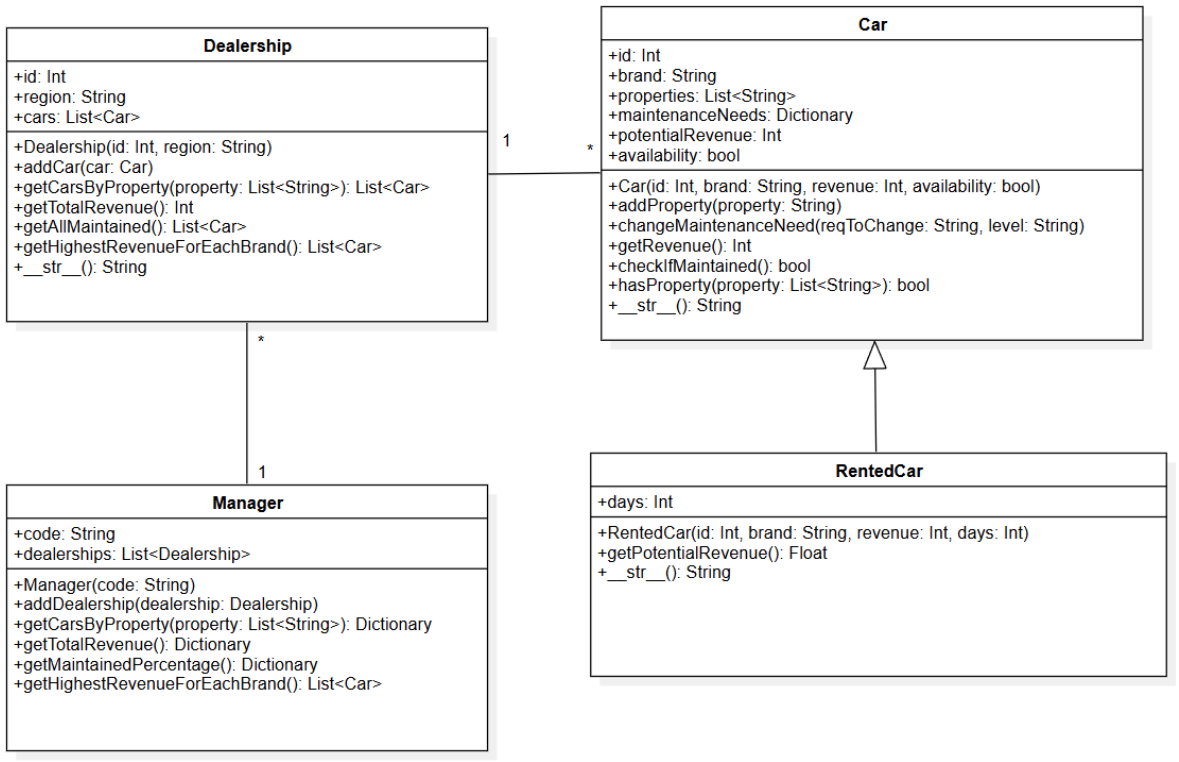


Figure 1: The Class Diagram of System

Car Class

Attribute	Explanation
id	The identification number for a car. It is unique for each car.
brand	The name of the car brand.
properties	Properties of a car that can be interesting for customers. It is in the form of a list of strings.
maintenanceNeeds	A dictionary that holds the severity of which parts of the car need to go through maintenance. The severity level can be None, Low, Medium or High. The parts, which will act as the keys in the dictionary, include; Engine, Tires, BodyShell, Frame and Interior. Unless otherwise specified in the data, these should be set to None.
potentialRevenue	Expected revenue for the dealership from selling the car.
availability	Should be True only if the car has not been rented or sold.

Method	Explanation
Car	Constructor that takes id, brand, revenue and availability.

addProperty	Adds the given property to the properties list.
changeMaintenanceNeed	From the dictionary, change the given maintenance need to the given severity.
getRevenue	Return the potential revenue attribute.
checkIfMaintained	Need to calculate a score. The score is calculated by assigning the numbers 0, 1, 2 and 3 to the None, Low, Medium, High for the maintenance needs and take the sum. If the resulting score is higher than or equal to 11, the car has not met the maintenance needs so it should return false. Otherwise return true.
hasProperty	Return true if the car has the given properties. The given property can be a single property or multiple.
__str__	Return a human readable string containing the car's id, brand, revenue, whether it was sold or not, maintenance needs and properties. The string may contain newlines.

RentedCar Class

Attribute	Explanation
days	Total days the car has been rented for.

Method	Explanation
RentedCar	Constructor that takes id, brand, revenue, and days.
getRevenue	Returns revenue divided by 40 times days, for example, if the car's revenue is 8000 and days is 30, then return (8000 / 40) * 30 = 6000
__str__	Return a human readable string containing the car's id, brand, revenue, whether it was rented and for how many days, maintenance needs and properties. The string may contain newlines.

Dealership Class

Attribute	Explanation
id	The identification number for a dealership. It is unique for each dealership.
region	A code that shows the region the dealership is in.
cars	List of all cars in the dealership that are available or sold/rented.

Method	Explanation
Dealership	Constructor that takes id and region.
addCar	Adds a car object to the cars list.
getCarsByProperty	Return all cars with the given properties.
getTotalRevenue	Get the sum of revenue from cars that have been sold or rented.
getAllMaintained	Get all cars that have met the maintenance needs.
getHighestRevenueForEachBrand	Returns the sold/rented cars with the highest revenue for each brand. If there are no cars that have been sold/rented for a specific brand then no car will be added to that list from that brand.

__str__	Return a human readable string containing the dealership's id and region. The string may contain newlines

Manager Class

Attribute	Explanation
code	Code unique to the manager
dealerShips	List of dealerships

Method	Explanation
Manager	Constructor that takes manager code.
addDealership	Adds a dealership object to the dealership list.
getCarsByProperty	Return a dictionary showing all cars from all dealerships that have the given properties. It should be clear which cars belong to which dealership from the dictionary. Dealership ID will be a key of the dictionary.
getTotalRevenue	Should return a dictionary that shows the total revenue **for each** dealership. Dealership ID will be a key of the dictionary.
getMaintainedPercentage	Return a dictionary that shows what percentage of the cars are maintained **for each** dealership. Dealership ID will be a key of the dictionary.
getHighestRevenueForEachBrand	Returns the sold/rented cars with the highest revenue for each brand **across all** the dealerships.

Main Program Operation

On startup, the application should take the filenames as a command-line argument. The application should read the given files to populate all the objects of the system. Then, the application should ask the user for a manager code. The application will only consider the dealerships managed by the given manager, if the manager code does not exist in the system an error message should be displayed. Otherwise, the application will give some options to the user:

1- Print Cars By Properties: The application should ask the user to enter properties separated by commas. Then the application should display for all dealerships the cars that have the properties. If no car with the properties exists an appropriate message should be printed.
2- Print Total Revenue: The application should display the total revenue for each dealership. This should be visualized with a pie chart (example given at the end of the document).
3- Print Maintenance Performance: For each dealership, the application should print the percentage of cars that have been well maintained.
4- Print Costliest Car For Each Brand: The application should print the highest revenue car that has been sold or rented across all the dealerships.

Rules

- You need to write your program using Python 3.x.
- You can only use built-in functions and modules (apart from matplotlib and pandas)
- You also need to create a file called ReadMe.txt which contains the following items. Please note that if you do not submit ReadMe.txt, your submission will not be evaluated.
    - Team members
    - Which version of Python 3.x you have used
    - Which operating system you have used
    - How you have worked as a team, especially how you have divided the tasks among the team members (who was responsible for what?), how you have communicated, how you have tested the program, etc.
- You need to put all your files into a folder which is named with your student id(s) and submit the compressed version of the folder in the .zip format.
- Only one team member should submit the assignment.
- Code quality, modularity, efficiency, maintainability, and appropriate comments will be part of the grading.

Grading Policy

Grading Item	Mark(out of 100)
Validating input file lines through Regular Expressions	15
Parsing data and Creation of the object structure	20
Using Pandas For Data Modification	15
Printing Cars With Given Properties	10
Printing Total Revenue and Chart	10
Printing Costliest Car For Each Brand	10
Printing Maintenance Performance	10
Taking command line arguments, reading files, and managing menu operations	5
Correct Implementation of Class Relations	5

Printing Charts

Printing Charts Provided below is a sample code to print a pie chart. Please note that you need to install matplotlib beforehand.

```
import matplotlib.pyplot as plt
dealershipIDs = ["101","102","103","104"]
revenues = [1000030, 1202000, 2300100, 952233]
plt.pie(revenues, labels=dealershipIDs)
plt.title('Total Revenue')
plt.show()
```

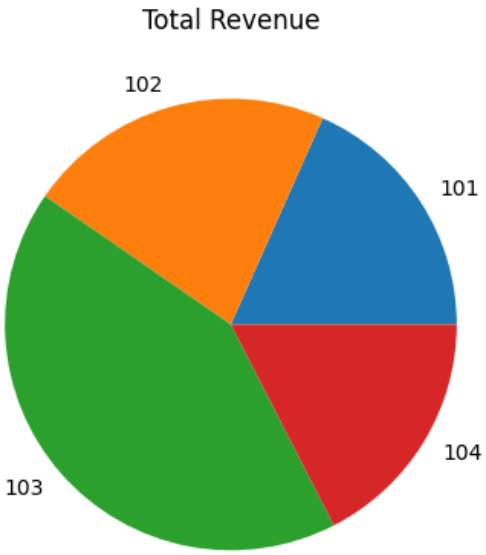


Figure 2: Sample Pie Chart

Sample Run

> python dealershipanalyser.py cars.txt dealership.txt

```
Error!! 66 does not match format and will be excluded:
1104;Toyota;LowBudget-Hybrid-ParkingCamera-LowKilometer-FuelEfficient;Interio
r:High;high;available;0;104

Error!! 87 does not match format and will be excluded:
1885;Nissan;RecentModel;Frame:High,Engine:Medium;7240;rented;24;10d

Error!! 88 does not match format and will be excluded:
1551;Mitsubishi;FuelEfficient-ParkingCamera;Engine:Low,BodyShell:High;10350;r
ented;22;1011

Error!! 104 does not match format and will be excluded:
10g7;Toyota;RecentModel-LowBudget-ParkingCamera-LowKilometer;Engine:Medium,Fr
ame:Low,Tires:Medium;11560;sold;0;105


Successfully validated and loaded data!

New Revenue to replace incorrect data: 8535

Successfully populated all objects.

Enter Manager Code: M105

Error: Manager code does not exist in the system.

Enter Manager Code: M101


Please select an option (1-5):

1. Print Cars By Properties
```

2. Print Total Revenue

3. Print Maintenance Performance

4. Print Costliest Car For Each Brand

5. Exit

Select an option (1-5): **1**

Enter properties separated by commas: **LowBudget, LowKilometer, FuelEfficient, ParkingCamera**

Dealership 101:

Car ID: 1014 | Brand: Toyota | Revenue: 10680 | Status: Sold

  Properties: ParkingCamera, FuelEfficient, LowKilometer, LowBudget, ParkingSensors

  Maintenance: {'Engine': 'Medium', 'Tires': 'High', 'BodyShell': 'High', 'Frame': 'High', 'Interior': 'Low'}

Car ID: 1041 | Brand: Toyota | Revenue: 6240 | Status: Sold

  Properties: FuelEfficient, LowBudget, LowKilometer, ParkingCamera

  Maintenance: {'Engine': 'None', 'Tires': 'None', 'BodyShell': 'None', 'Frame': 'Low', 'Interior': 'High'}

Dealership 102:

RentedCar ID: 1006 | Brand: Nissan | Base Revenue: 8535 | Rented for: 2 days

  Properties: FuelEfficient, ParkingCamera, LowKilometer, ParkingSensors, LowBudget, RecentModel

  Maintenance: {'Engine': 'High', 'Tires': 'Medium', 'BodyShell': 'High', 'Frame': 'None', 'Interior': 'High'}

Car ID: 1058 | Brand: Nissan | Revenue: 10280 | Status: Available

  Properties: LowKilometer, FuelEfficient, ParkingCamera, RecentModel, LowBudget, ParkingSensors

  Maintenance: {'Engine': 'None', 'Tires': 'Medium', 'BodyShell': 'None', 'Frame': 'None', 'Interior': 'Medium'}

Car ID: 1088 | Brand: Toyota | Revenue: 8535 | Status: Sold

  Properties: LowBudget, Hybrid, ParkingCamera, LowKilometer, RecentModel, FuelEfficient

  Maintenance: {'Engine': 'Low', 'Tires': 'High', 'BodyShell': 'None', 'Frame': 'Medium', 'Interior': 'High'}

Dealership 103:

Car ID: 1001 | Brand: Nissan | Revenue: 6760 | Status: Available

Properties: Hybrid, FuelEfficient, LowBudget, ParkingCamera, LowKilometer

Maintenance: {'Engine': 'High', 'Tires': 'High', 'BodyShell': 'Low', 'Frame': 'High', 'Interior': 'Medium'}

Car ID: 1054 | Brand: Toyota | Revenue: 8535 | Status: Available

Properties: ParkingCamera, LowBudget, ParkingSensors, FuelEfficient, LowKilometer

Maintenance: {'Engine': 'None', 'Tires': 'High', 'BodyShell': 'None', 'Frame': 'None', 'Interior': 'Low'}

Dealership 104:

Car ID: 1033 | Brand: Toyota | Revenue: 8535 | Status: Available

Properties: ParkingCamera, LowKilometer, LowBudget, ParkingSensors, RecentModel, FuelEfficient

Maintenance: {'Engine': 'Medium', 'Tires': 'High', 'BodyShell': 'High', 'Frame': 'Medium', 'Interior': 'High'}

Car ID: 1075 | Brand: Toyota | Revenue: 7920 | Status: Available

Properties: ParkingSensors, ParkingCamera, LowKilometer, RecentModel, FuelEfficient, LowBudget

Maintenance: {'Engine': 'None', 'Tires': 'High', 'BodyShell': 'Medium', 'Frame': 'None', 'Interior': 'None'}

Please select an option (1-5):

1. Print Cars By Properties

2. Print Total Revenue

3. Print Maintenance Performance

4. Print Costliest Car For Each Brand

5. Exit

Select an option (1-5): **3**

Maintenance Performance:

Dealership 101: 75.00% of cars are well maintained.

Dealership 102: 100.00% of cars are well maintained.

Dealership 103: 91.67% of cars are well maintained.

Dealership 104: 87.50% of cars are well maintained.

Please select an option (1-5):

1. Print Cars By Properties

2. Print Total Revenue

3. Print Maintenance Performance

4. Print Costliest Car For Each Brand

5. Exit

Select an option (1-5): **4**

Costliest Sold/Rented Car For Each Brand:

Car ID: 1048 | Brand: Toyota | Revenue: 12610 | Status: Sold

  Properties: RecentModel, ParkingSensors

  Maintenance: {'Engine': 'Low', 'Tires': 'None', 'BodyShell': 'High', 'Frame': 'Low', 'Interior': 'None'}

Car ID: 1028 | Brand: Nissan | Revenue: 11920 | Status: Sold

  Properties: FuelEfficient, RecentModel, ParkingSensors, Hybrid, LowKilometer

  Maintenance: {'Engine': 'None', 'Tires': 'Medium', 'BodyShell': 'None', 'Frame': 'None', 'Interior': 'Low'}

Car ID: 1025 | Brand: Mitsubishi | Revenue: 10700 | Status: Sold

  Properties: LowKilometer, RecentModel, ParkingSensors

  Maintenance: {'Engine': 'Low', 'Tires': 'Medium', 'BodyShell': 'High', 'Frame': 'Low', 'Interior': 'Medium'}

Please select an option (1-5):

1. Print Cars By Properties

2. Print Total Revenue

3. Print Maintenance Performance

4. Print Costliest Car For Each Brand

5. Exit

Select an option (1-5): **5**